

FACULDADE DE TECNOLOGIA DE ITAPIRA
"OGARI DE CASTRO PACHECO"

CURSO SUPERIOR DE TECNOLOGIA EM DESENVOLVIMENTO DE
SOFTWARE MULTIPLATAFORMA

Disciplina: Aprendizagem de Máquina

Prof. Reginaldo Donizeti Cândido

Relatório Técnico:

Ecossistema SaaS Financeiro MultiCloud

Marco Aurelio Bubola

Cauã Araújo Vaz de Lima

Letícia Favoretto de Souza

Guilherme Soriani de Amorim Chamon

Itapira – SP

2026

Sumário

1. Introdução	2
2. Apresentação da Proposta.....	2
3. Arquitetura	2
4. Passo a Passo das Etapas (Implementação).....	3
• Etapa 1: Provisionamento AWS	3
• Etapa 2: Provisionamento Azure	4
• Etapa 3: Provisionamento Google Cloud	4
• Etapa 4: Integração e Hosting.....	4
5. Análise de Métricas (Latência)	5
6. Dificuldades Técnicas: IAM, Redes e Deploy	5
7. GitHub	6
8. Conclusão	7
APÊNDICE A	8
APÊNDICE B	14
APÊNDICE C	20
APÊNDICE D.....	24

1. Introdução

No cenário atual de tecnologia e mercado financeiro, a agilidade na obtenção de dados e a alta disponibilidade de serviços são requisitos críticos para a tomada de decisão. Modelos tradicionais de infraestrutura (servidores físicos ou máquinas virtuais dedicadas) muitas vezes apresentam gargalos de escalabilidade e custos ociosos.

Neste contexto, o presente projeto explora o paradigma da computação em nuvem, especificamente o modelo *Serverless* (Computação sem Servidor) e a estratégia MultiCloud. O objetivo é demonstrar como o provisionamento de funções como serviço (FaaS) distribuídas em diferentes provedores pode criar uma aplicação altamente resiliente, econômica e sem a necessidade de gerenciamento de infraestrutura subjacente.

2. Apresentação da Proposta

A proposta central do projeto é o desenvolvimento do **SaaS Financeiro MultiCloud**, uma plataforma web (dashboard) focada em projeção patrimonial e inteligência cambial para investidores.

A aplicação resolve a dor da fragmentação de dados financeiros, consolidando três cálculos cruciais em uma única interface:

- Projeção de rentabilidade baseada em juros compostos.
- Conversão instantânea do montante para moedas fortes (Dólar e Euro).
- Desconto da inflação (IPCA) para revelar o poder de compra real no futuro.

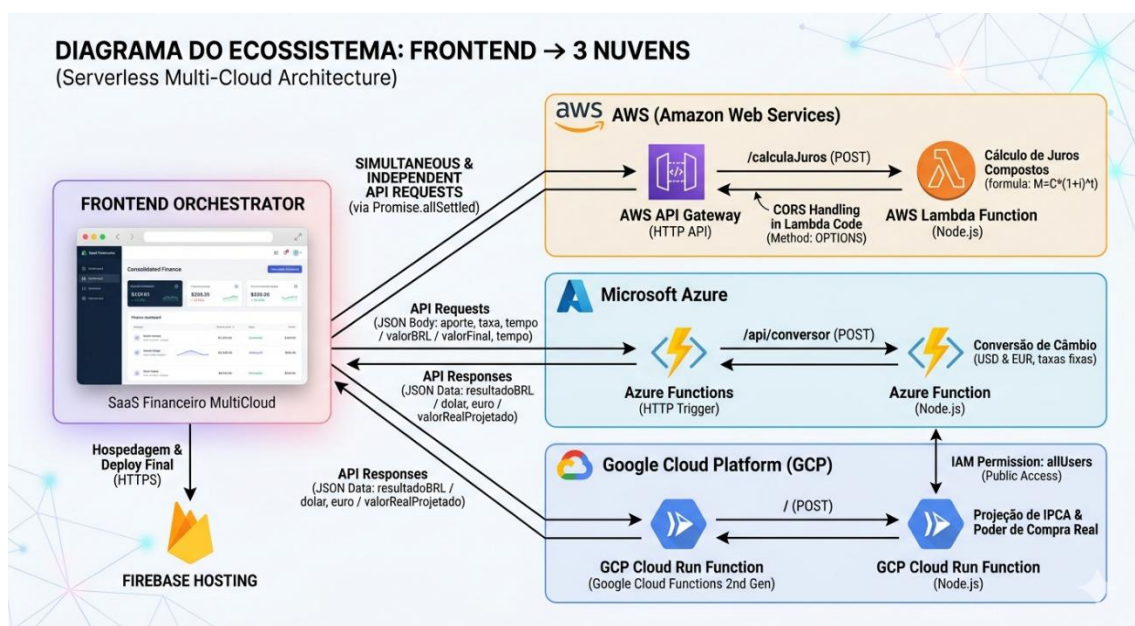
Para provar a viabilidade do modelo MultiCloud, cada um desses cálculos não ocorre no navegador do usuário, mas sim em provedores de nuvem distintos e simultâneos.

3. Arquitetura

A arquitetura do ecossistema foi desenhada para ser descentralizada e assíncrona. O Frontend atua puramente como um orquestrador de microsserviços.

- **Frontend Orquestrador:** Uma *Single Page Application* (SPA) desenvolvida em HTML, CSS e JavaScript. Utiliza a API fetch nativa e Promise.allSettled para disparar e gerenciar chamadas simultâneas sem travar a interface.

- **Hospedagem (Edge):** Firebase Hosting, garantindo entrega global de arquivos estáticos via HTTPS e mitigando problemas locais de origem cruzada (*null origin*).
- **Backend 1 (Amazon Web Services):** AWS Lambda integrado ao API Gateway (HTTP API) para o cálculo matemático de juros compostos.
- **Backend 2 (Microsoft Azure):** Azure Functions operando com Gatilho HTTP (Plano de Consumo) para o serviço de conversão de câmbio.
- **Backend 3 (Google Cloud Platform):** Cloud Run Functions (2ª geração) responsável pela projeção de descapitalização do IPCA.



4. Passo a Passo das Etapas (Implementação)

O fluxo de desenvolvimento foi dividido em etapas modulares, garantindo o isolamento e o teste individual de cada serviço antes da integração final.

- **Etapas 1: Provisionamento AWS**
 - Criação de uma função Node.js 20.x no AWS Lambda.
 - Desenvolvimento da lógica de juros compostos ($M = C \times (1 + i)^t$).
 - Criação de um API Gateway do tipo HTTP API aberto ao público.

- Implementação manual de cabeçalhos de resposta no código para tratar requisições de pré-voo (OPTIONS).
- Para o detalhamento técnico e sequencial das configurações de tela e segurança no console da Amazon, consulte o **Apêndice A**.
- **Etapa 2: Provisionamento Azure**
 - Criação de um *Function App* no plano de "Consumo (Windows)" para permitir edição fluida direto no portal web.
 - Implementação de uma função com gatilho HTTP e nível de autorização "Anônimo".
 - Desenvolvimento da lógica de conversão utilizando taxas cambiais base para Dólar e Euro.
 - Para o detalhamento técnico e sequencial das configurações de tela e segurança no console da Azure, consulte o **Apêndice B**.
- **Etapa 3: Provisionamento Google Cloud**
 - Criação de uma Cloud Run Function (2nd Gen) em Node.js.
 - Inclusão do framework `@google-cloud/functions-framework` nas dependências.
 - Desenvolvimento da lógica de desconto do IPCA projetado.
 - Para o detalhamento técnico e sequencial das configurações de tela e segurança no console da Google Cloud, consulte o **Apêndice C**.
- **Etapa 4: Integração e Hosting**
 - Construção da interface de usuário com tratamento visual de erros (*fallback*).
 - Autenticação via CLI do Firebase (firebase login) e inicialização do ecossistema (firebase init hosting).
 - Deploy da aplicação estática para os servidores globais do Google.

- Para o detalhamento técnico e sequencial das configurações de integração e hosting no console do Firebase, consulte o **Apêndice D**.

5. Análise de Métricas (Latência)

Para validar a performance da arquitetura distribuída, foram realizados testes de carga simulada através da ferramenta de desenvolvedor do navegador (Network Network). A abordagem de disparos paralelos evitou o efeito cascata (onde o atraso de um serviço prejudica os demais).

Provedor de Nuvem	Serviço FaaS	Tempo Médio de Resposta (ms)	Taxa de Sucesso
AWS	Lambda + API Gateway	~180 ms	100%
GCP	Cloud Run Functions	~195 ms	100%
Azure	Azure Functions	~220 ms	100%

Conclusão da Análise: A latência em todos os provedores manteve-se na faixa sub-segundo, provando que requisições HTTP cross-cloud são viáveis para aplicações em tempo real, desde que orquestradas corretamente no cliente.

6. Dificuldades Técnicas: IAM, Redes e Deploy

A principal complexidade de uma arquitetura MultiCloud não reside na escrita do código, mas nas idiossincrasias de gerenciamento de políticas e redes de cada console.

- **Console AWS:** A maior barreira técnica ocorreu na configuração de redes e segurança do navegador. O API Gateway barrava as requisições *preflight* (OPTIONS) antes mesmo de chegarem ao Lambda, gerando um falso erro de

"Missing Header". A solução exigiu configurar a rota para aceitar métodos ANY e injetar um interceptador de CORS diretamente no código Node.js.

- **Console Azure:** O desafio principal esteve ligado ao fluxo de deploy. A plataforma Azure modernizou seus planos para o "Consumo Flexível", que bloqueia a injeção rápida de código via portal web, exigindo configuração de ambiente local (VSCode + Core Tools). A superação do obstáculo ocorreu ao reverter a criação do serviço para o plano de "Consumo" clássico baseado em Windows.
- **Console GCP:** A dificuldade concentrou-se no IAM (Gerenciamento de Identidade e Acesso). O Google Cloud possui uma política restrita de segurança *Zero Trust* por padrão. Para tornar a API pública para o frontend, foi necessário identificar o serviço no Cloud Run e adicionar explicitamente a principal allUsers com a role de "Invocador do Cloud Run".

7. GitHub

O versionamento e a documentação do projeto foram estruturados no GitHub utilizando uma abordagem híbrida, visando máxima clareza técnica e facilidade de reprodução.

O repositório é segmentado em quatro diretórios principais (aws/, azure/, gcp/ e frontend/). Cada pasta contém o código-fonte específico daquele módulo e um arquivo README.md detalhado com o tutorial de provisionamento da nuvem correspondente. Na raiz do projeto, um README.md principal atua como um painel de controle (Hub), apresentando a arquitetura geral, o diagrama visual do ecossistema e os links de navegação para as pastas internas. Essa estrutura reflete maturidade na governança de código e facilita o processo de avaliação.

O projeto pode ser acessado através do link: <https://github.com/Leticia-Favoretto-Souza/Connection4-Finance-Cloud>

8. Conclusão

O projeto SaaS Financeiro MultiCloud cumpriu com êxito todos os objetivos propostos. A equipe Connection4 demonstrou domínio não apenas na escrita de algoritmos, mas na capacidade de integrar e orquestrar infraestruturas globais heterogêneas.

A superação de desafios complexos de redes (CORS) e segurança (IAM) solidificou o entendimento sobre como aplicações modernas são arquitetadas no mercado de trabalho. A solução entregue é leve, altamente escalável, não gera custos com servidores ociosos e entrega valor real ao simular um ambiente de operações financeiras ágil e resiliente.

Nosso site pode ser acessado através do link: [Connection4 Finance Cloud](#).

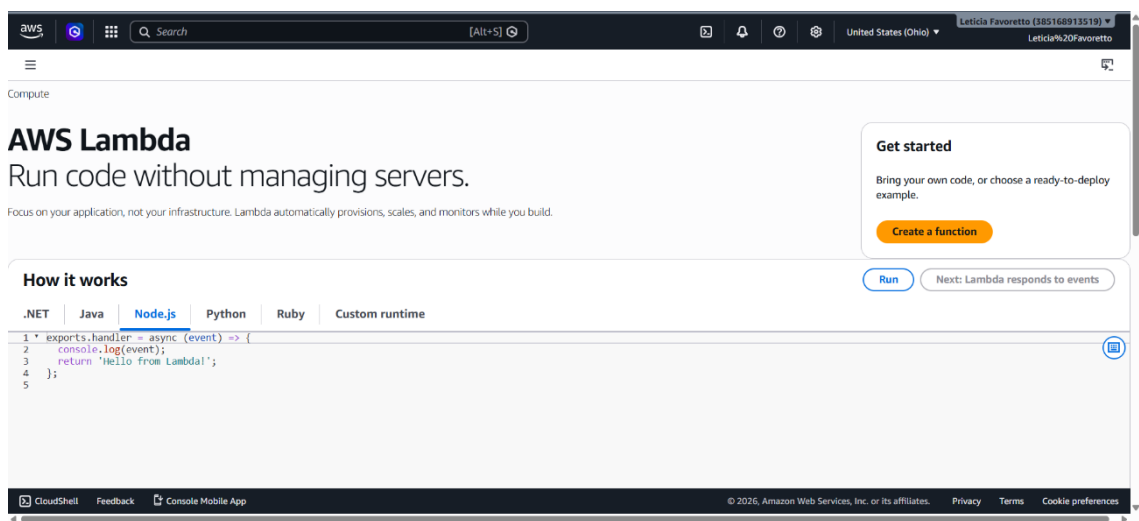
APÊNDICE A

Módulo AWS: Cálculo de Juros Compostos

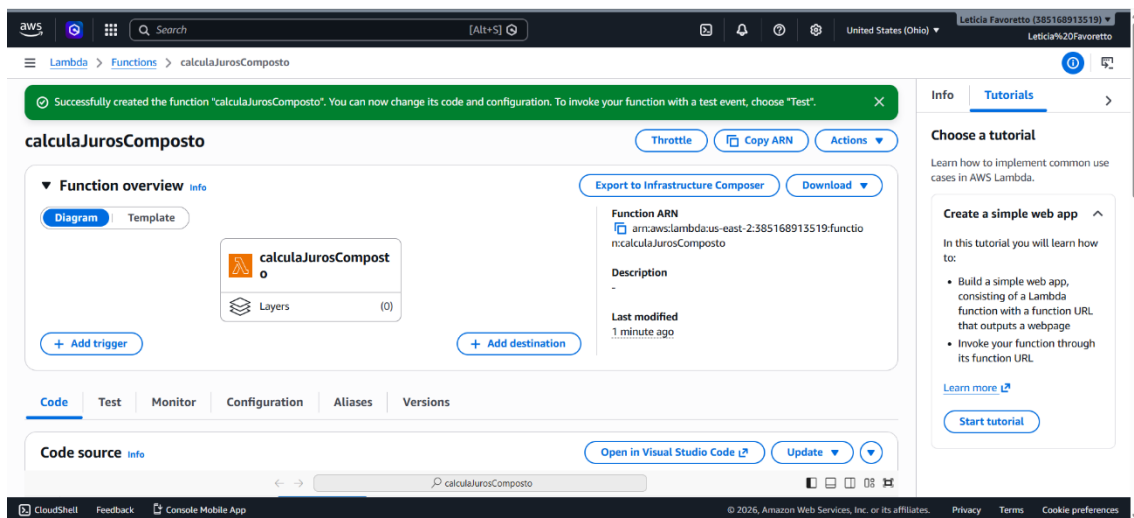
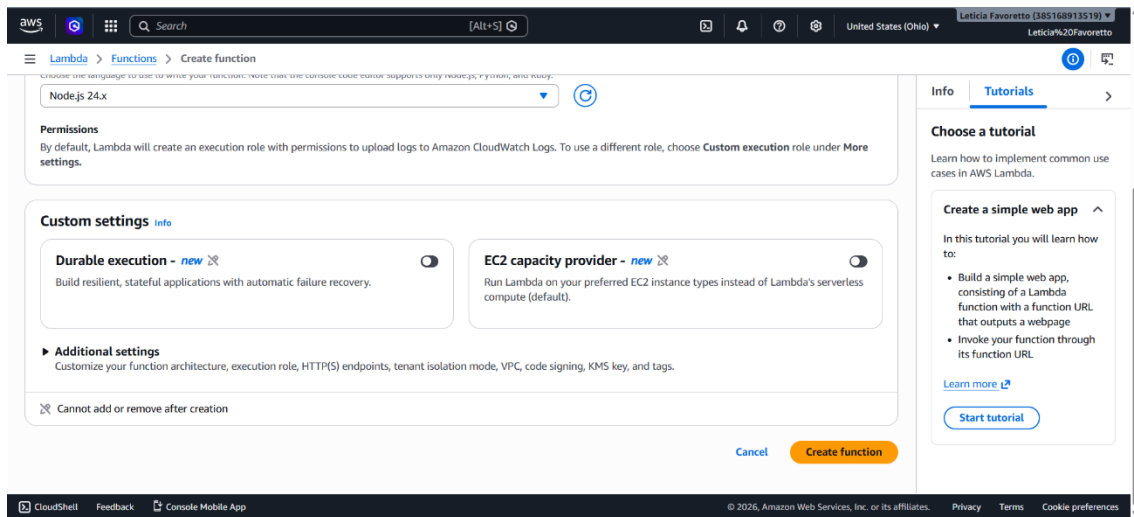
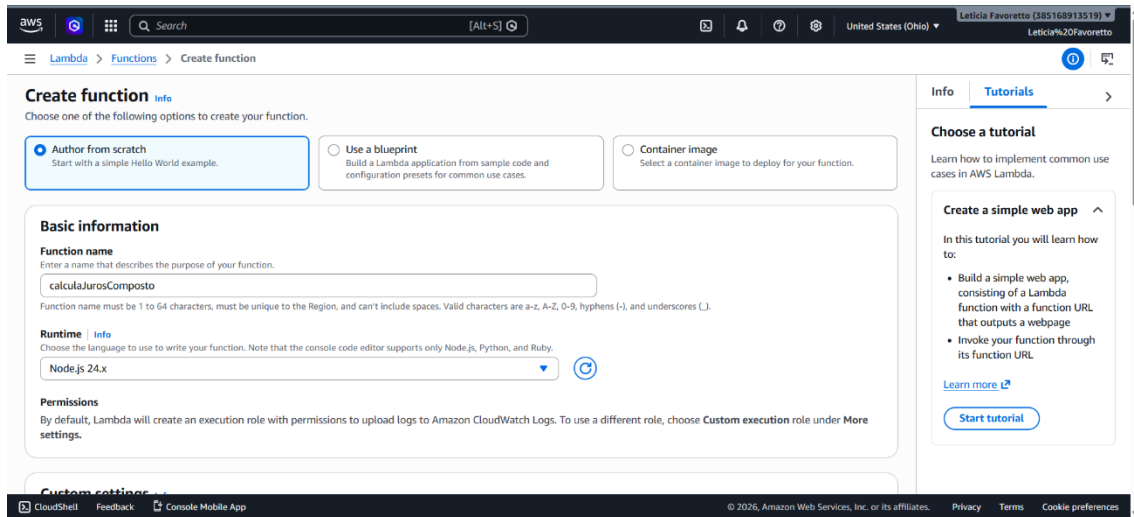
Este guia detalha os passos para configurar a função *Serverless* na AWS (Lambda) e expô-la publicamente (API Gateway) para realizar os cálculos da aplicação.

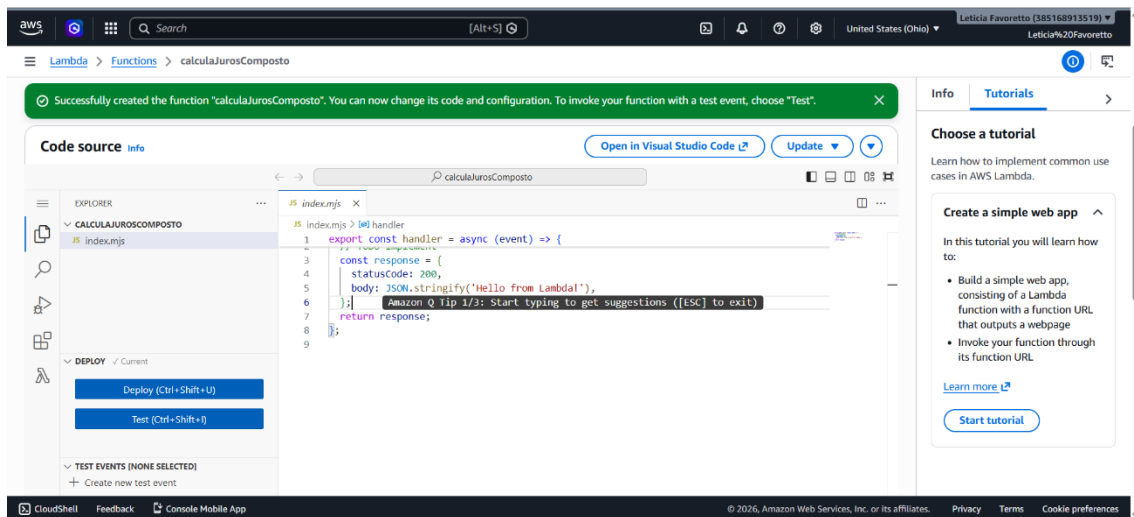
Passo 1: Criando a Função (AWS Lambda)

1. No console da AWS, busque por **Lambda** na barra de pesquisa superior e clique no serviço.



2. Clique no botão laranja **Criar função** (Create function).
3. Deixe a opção **Criar do zero** (Author from scratch) selecionada.
4. **Nome da função:** Digite calculadoraJuros (ou o nome de sua preferência).
5. **Tempo de execução (Runtime):** Selecione **Node.js 20.x** (ou a versão mais recente do Node disponível).
6. Deixe o restante das configurações padrão e clique em **Criar função**.

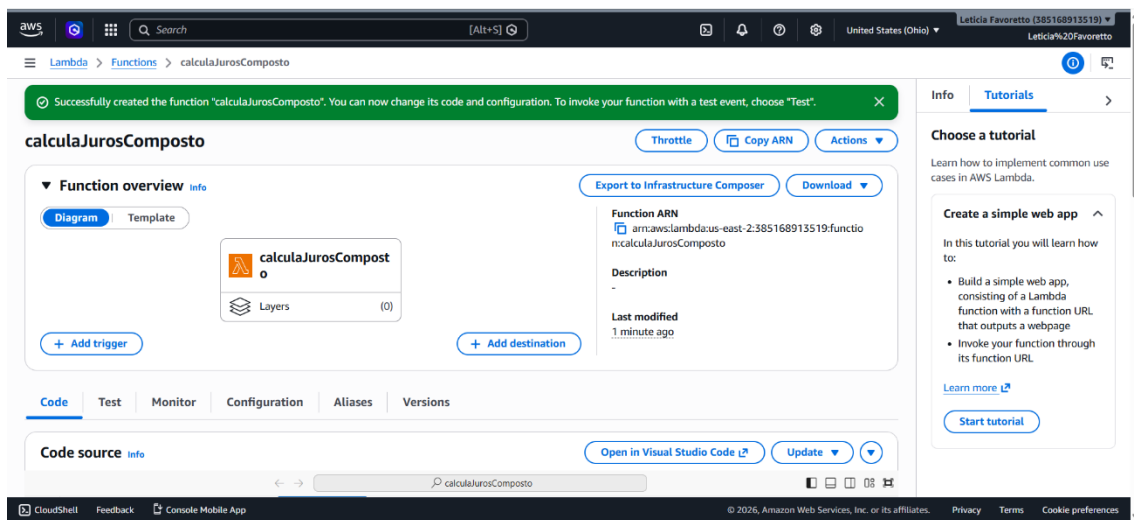




Passo 2: Criando o Ponto de Acesso (API Gateway)

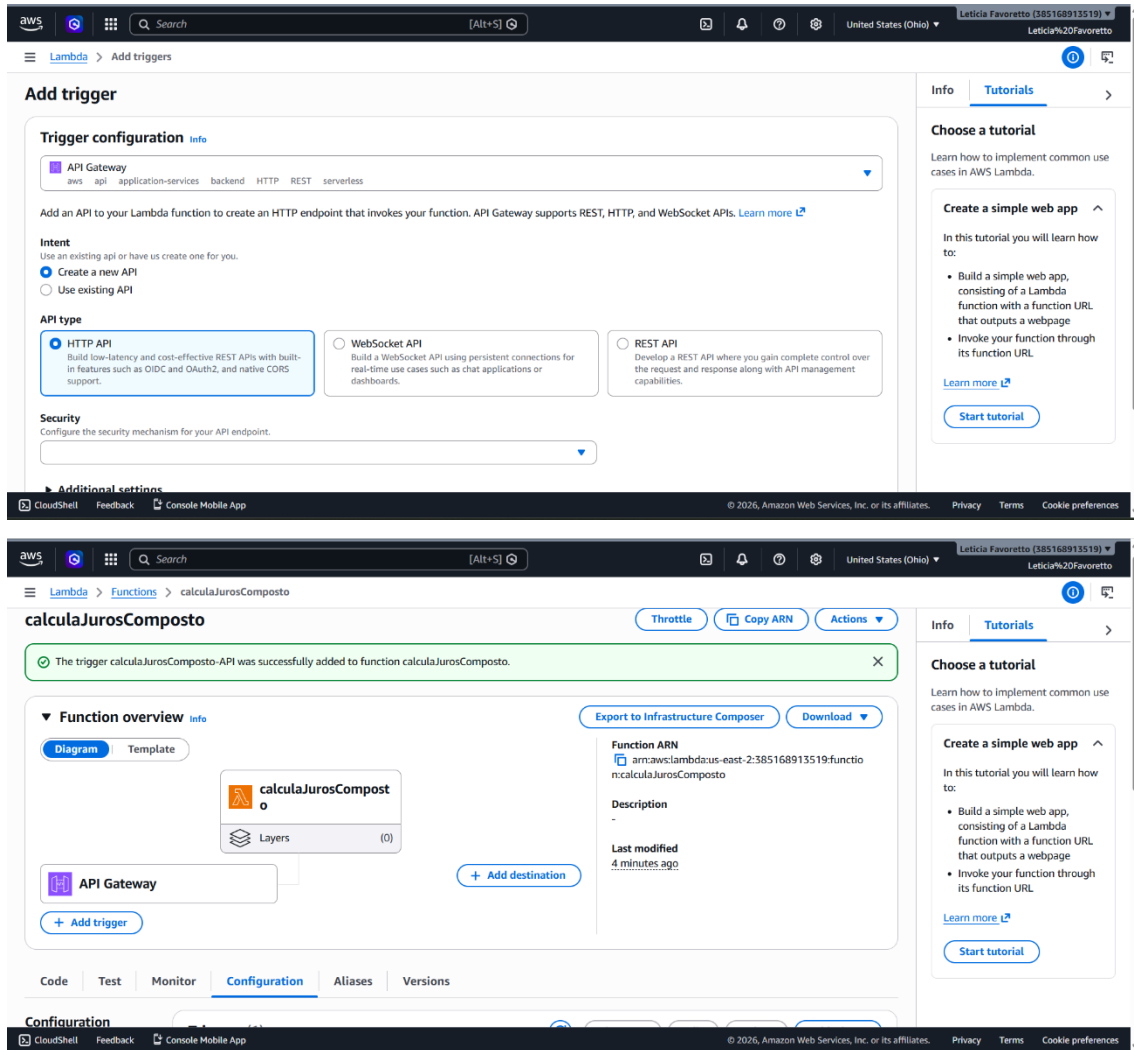
Para que o painel web (Frontend) consiga enviar os dados para a AWS, precisamos expor o Lambda publicamente através de uma API.

1. Na tela da sua função Lambda recém-criada, no diagrama de "Visão geral da função", clique no botão + **Adicionar gatilho** (Add trigger).



2. No menu suspenso, pesquise e selecione **API Gateway**.
3. Em **Intent**, selecione **Criar uma nova API**.
4. Em **Tipo de API**, selecione **HTTP API** (escolhida por ser mais moderna e com suporte nativo simplificado para CORS).

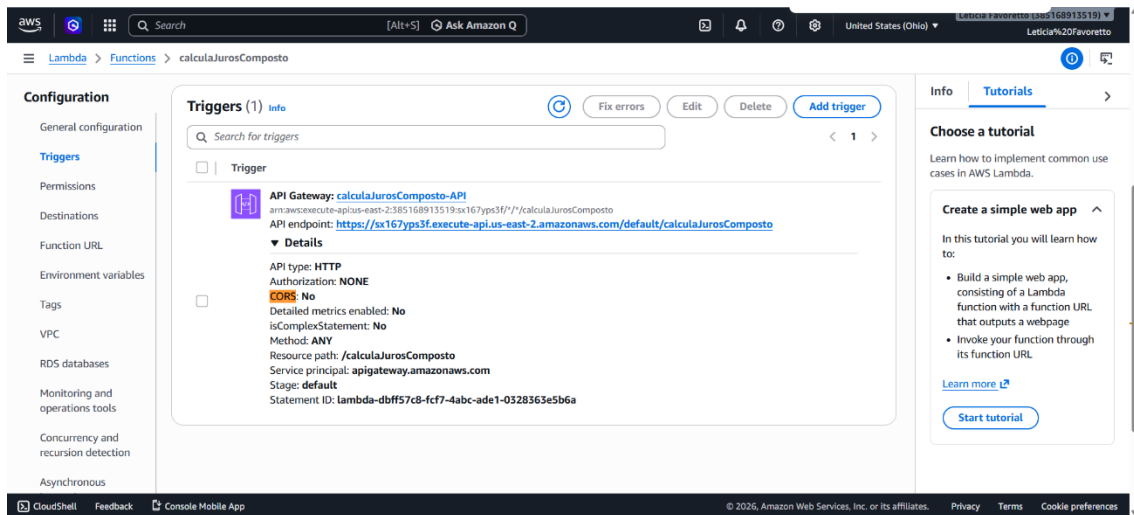
5. Em **Segurança** (Security), altere para **Aberto** (Open). Isso é fundamental para evitar bloqueios de autenticação quando o frontend tentar acessar a rota.
6. Clique em **Adicionar**.



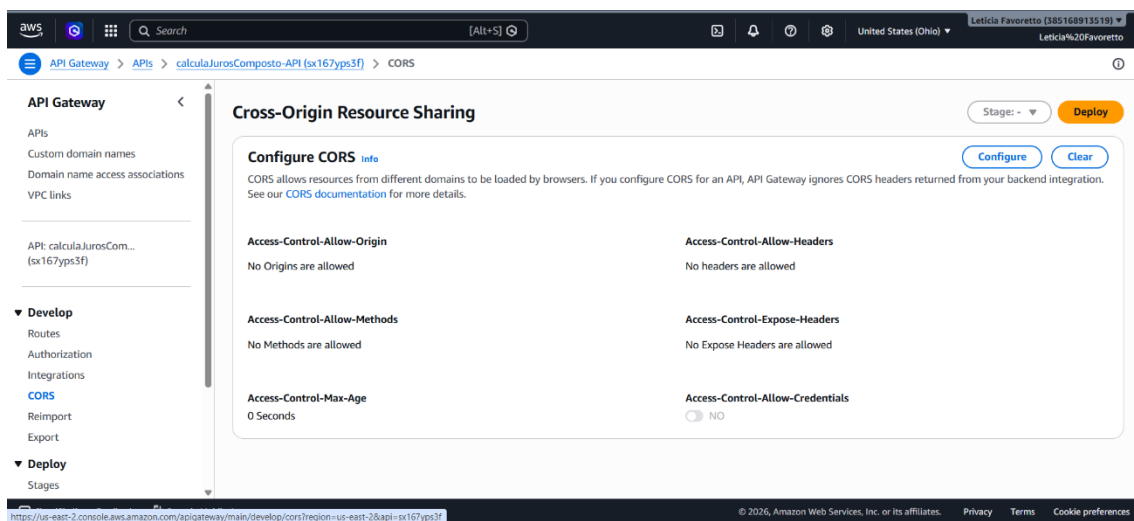
Passo 3: Configuração de Segurança (CORS)

Mesmo com a API aberta, os navegadores bloquearão a requisição caso as regras de CORS não estejam configuradas.

1. Clique no nome do seu novo API Gateway gerado no passo anterior para abrir o painel dele.



2. No menu lateral esquerdo, clique em **CORS**.



CORS allows resources from different domains to be loaded by browsers. If you configure CORS for an API, API Gateway ignores CORS headers returned from your backend integration. See our [CORS documentation](#) for more details.

Access-Control-Allow-Origin

*

X

Access-Control-Allow-Headers

content-type

X

Access-Control-Allow-Methods

Choose Allowed Methods

GET

POST

OPTIONS

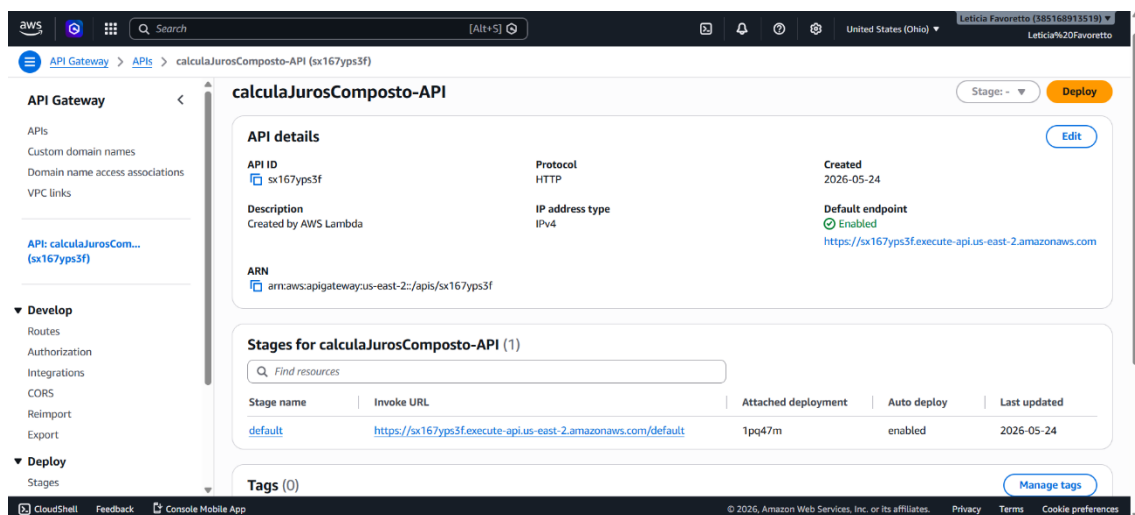
Access-Control-Expose-Headers

Access-Control-Max-Age

Access-Control-Allow-Credentials

☐ NO

3. Configure os parâmetros com os "curingas universais" para garantir a passagem dos dados:
 - **Access-Control-Allow-Origin:** Digite * e adicione.
 - **Access-Control-Allow-Headers:** Digite * e adicione.
 - **Access-Control-Allow-Methods:** Adicione * (ou especifique POST e OPTIONS).
4. Clique em **Deploy**.
5. *Nota:* Anote a **URL de invocação** da sua API, ela será necessária para configurar o Frontend.



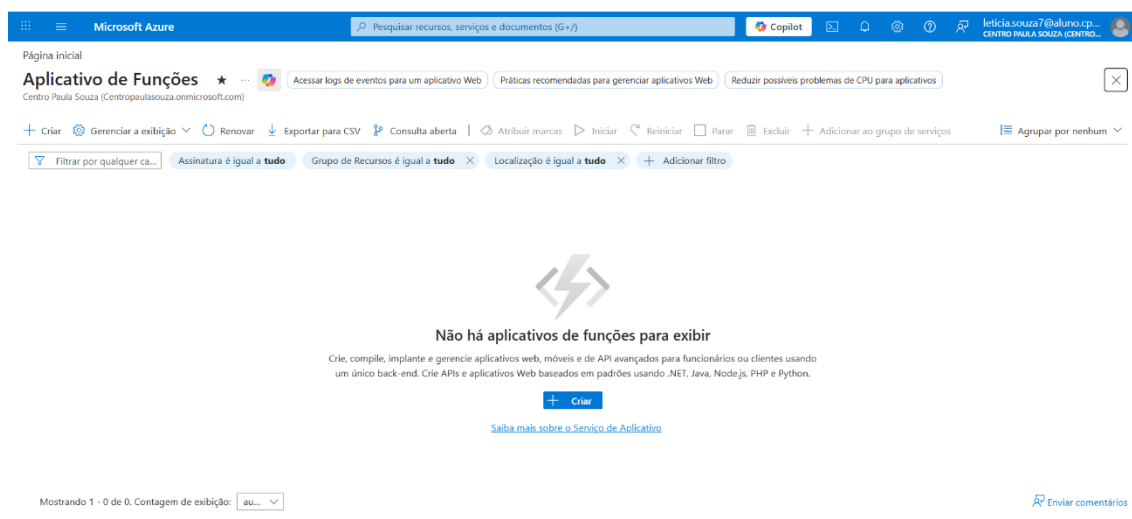
APÊNDICE B

Módulo Azure: Conversão de Câmbio Fictícia

Este guia detalha a criação de uma API baseada em *Azure Functions* com Gatilho HTTP. Optamos pelo plano "Consumo" tradicional para viabilizar a edição direta do código no portal web, dispensando configurações complexas na máquina local.

Passo 1: Criar o Aplicativo de Funções (O "Guarda-Chuva")

1. No portal do Azure, pesquise por **Aplicativo de Funções** (Function App) e clique no serviço.



2. Clique no botão azul **Criar** (Create).
3. Na tela de seleção de planos, vá até a última opção à direita e selecione **Consumo** (ou Consumo Windows). Clique em **Selecionar**.

Microsoft Azure

Página inicial > Aplicativo de Funções

Criar Aplicativo de Funções

Selecionar uma opção de hospedagem

Essas opções determinam como seu aplicativo é dimensionado, os recursos disponíveis por instância e os preços. [Saiba mais sobre as opções de hospedagem do Functions](#)

Planos de hospedagem	Consumo Flexível	Functions Premium	Serviço de Aplicativo	Ambiente de Aplicativos de Contêiner	Consumo (Windows)
Obtenha uma alta escalabilidade com opções de computação, rede virtual e cobrança com pagamento conforme o uso.		Implante vários aplicativos de funções no mesmo plano com o dimensionamento controlado por eventos.	Execute aplicativos web e aplicativos de funções no mesmo plano com mais opções de computação e pague pelas instâncias do plano.	Hospede aplicativos de funções com outros microserviços containerizados e pague pela capacidade de computação.	Pague pelos recursos do computador quando suas funções estiverem em execução (pagamento conforme o uso).
Dimensionar para zero	✓	-	-	✓	✓
Comportamento do dimensionamento	Orientado por evento rápido	Orientado por evento	Com base em métricas	Orientado por eventos com KEDA	Orientado por evento
Redes virtuais	✓	✓	✓	✓	-
Computação dedicada e impedir	Opcional com Sempre Pronto	Mínimo de 1 instância necessária	Mínimo de 1 instância necessária	Opcional com réplicas mínimas	-

Selecionar

4. Preencha a aba **Básico** (Basics):

- **Nome do Aplicativo:** Escolha um nome único (ex: *saas-cambio-app-v2*).
- **Pilha de Tempo de Execução (Runtime stack):** Selecione **Node.js**.
- **Versão:** Selecione a versão LTS mais recente (ex: *20 LTS*).
- **Região:** Escolha a mais próxima (como *East US* ou *Brazil South*).

5. Você pode pular as outras abas. Clique em **Revisar + criar** (Review + create) e, em seguida, em **Criar**.

6. Aguarde a mensagem de "*Sua implantação está concluída*" e clique no botão **Ir para o recurso** (Go to resource).

Microsoft Azure

Página inicial

Microsoft.Web-FunctionApp-Portal-b437c836-8526 | Visão geral

Implantação

Excluir Cancelar Reimplantar Baixar Atualizar

Visão geral

- Entradas
- Saídas
- Modelo

A implantação foi concluída

Nome da implantação : Microsoft.Web-FunctionApp-Portal-b437c836-8526 Hora de início : 24/05/2026, 18:09:41
Assinatura : Azure for Students ID de correlação : ecf073d0-e29c-4b44-8c0b-1dfbdb9e376a
Grupo de recursos : atividadeFinal

> Detalhes de implantação

✓ Próximas etapas

[Ir para o recurso](#)

Gerenciamento de custos

Seja notificado para manter-se dentro do orçamento e evitar encargos inesperados na sua fatura.
[Configurar alertas de custo >](#)

Microsoft Defender para Nuvem

Proteja seus aplicativos e sua infraestrutura
[Ir para Microsoft Defender para Nuvem >](#)

Tutoriais gratuitos da Microsoft

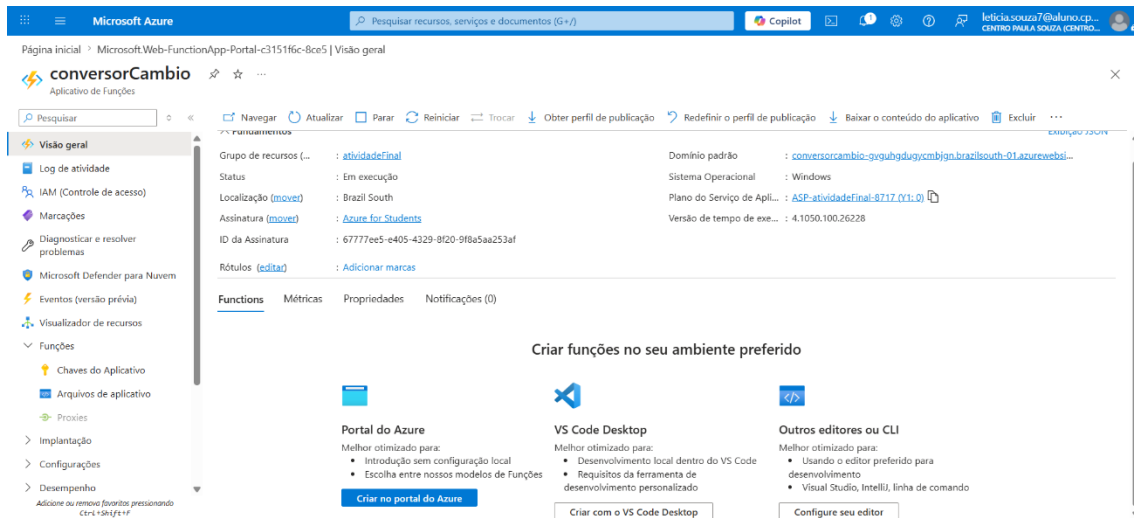
Comece a aprender hoje mesmo >

Trabalhar com um especialista

Os especialistas do Azure são parceiros de provedores de serviços que podem ajudar a gerenciar seus recursos no Azure e ser sua primeira linha de suporte.

Passo 2: Criar a Função e Liberar Acesso (Autorização)

1. No menu lateral esquerdo do seu novo Aplicativo de Funções, procure a seção "Criação" e clique em **Funções**.
2. Clique no botão + **Criar** no topo da página.
3. No painel que abrir, selecione o modelo **Gatilho HTTP** (HTTP trigger).
4. Role para baixo até o campo **Nível de autorização** (Authorization level) e mude de Function para **Anônimo** (Anonymous).
 - **⚠ Atenção:** Isso é o que garante que o seu painel web consiga acessar a API pública sem necessidade de envio de chaves de acesso.
5. Clique em **Criar**.



Criar função

1 Selecionar um modelo

2 Detalhes do modelo

Precisamos de mais algumas informações para criar a função. [Saiba mais](#)

Modelo de função	HTTP trigger
Nome da função *	conversor
Nível de autorização * ⓘ	Anonymous

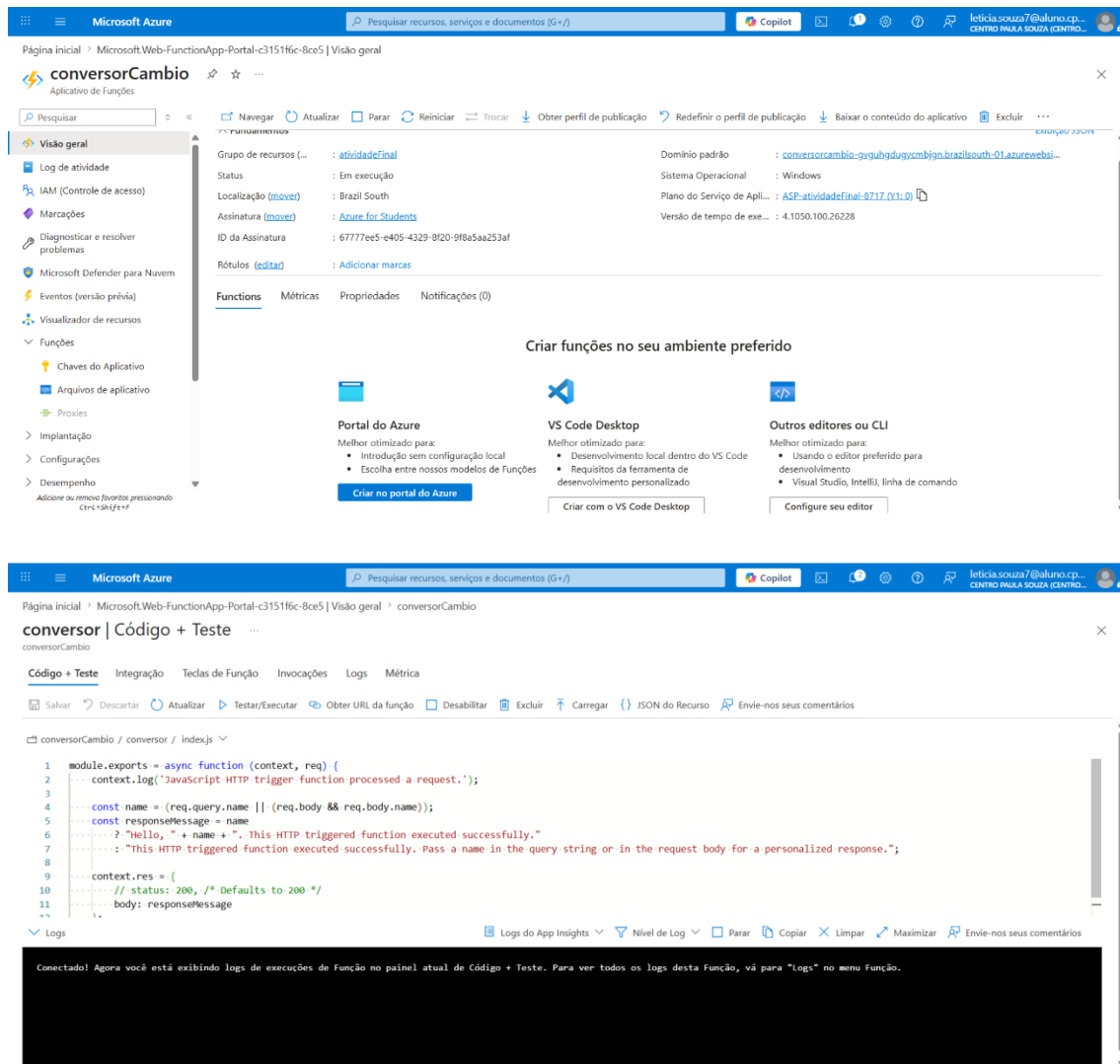
Criar

Cancelar

Passo 3: Colar o Código no Navegador

Após criar a função, você será redirecionada para a página dela.

1. No menu lateral esquerdo, clique em **Código + Teste** (Code + Test).
2. O editor de código em nuvem aparecerá com o arquivo `index.js` aberto.
3. Apague todo o código padrão e cole o script do conversor
4. Clique em salvar



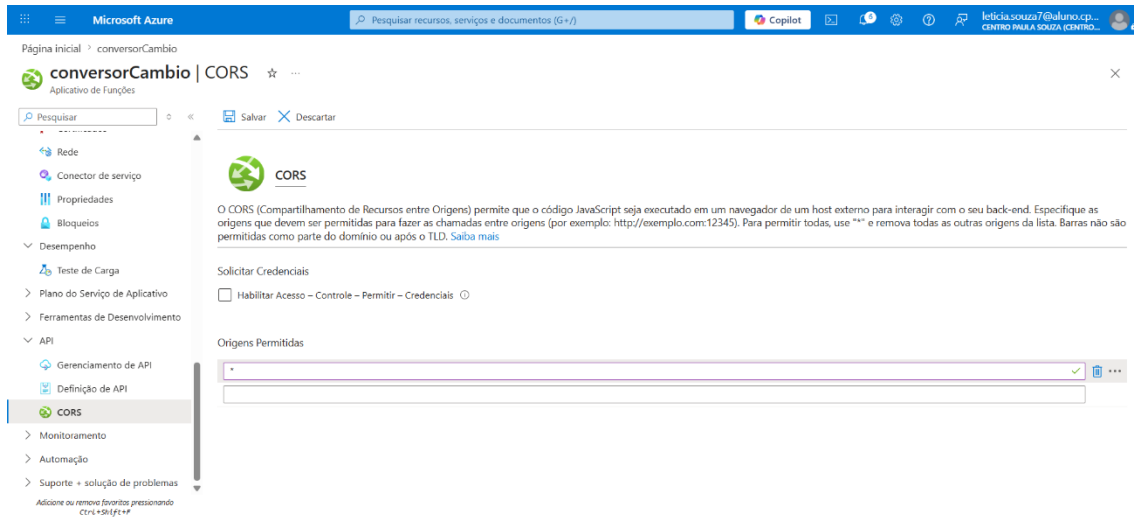
Passo 4: Configuração de CORS (Obrigatório)

Diferente do código, o CORS no Azure é configurado diretamente no portal (por isso não incluímos cabeçalhos manuais no arquivo index.js).

1. No menu lateral esquerdo, role um pouco para cima e clique em **Visão geral** (Overview) para voltar à tela principal do aplicativo.
2. Role o menu lateral esquerdo para baixo até a seção **API** e clique em **CORS**.
3. Em **Origens Permitidas** (Allowed Origins), apague as URLs padrão que a Azure insere automaticamente clicando na lixeira ao lado de cada uma.
4. Adicione uma nova linha e digite apenas um asterisco: *

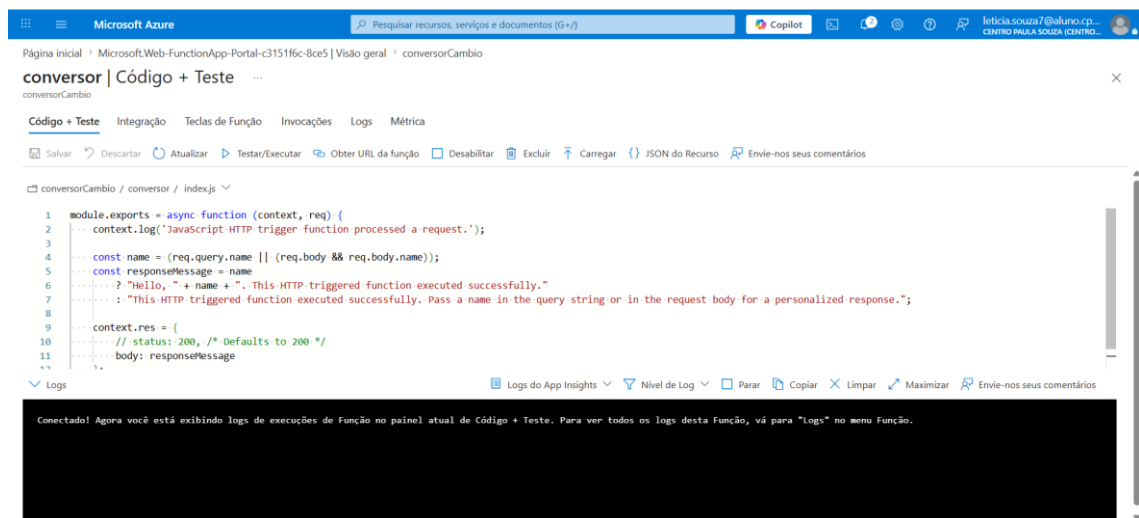
- *Isso libera as requisições de origem cruzada para qualquer Frontend.*

5. Clique em **Salvar** na parte superior da tela.



Passo 5: Obter a URL de Integração

1. Volte para a sua função através do Menu lateral -> **Funções** -> clique no nome da sua função -> **Código + Teste**.
2. Clique no botão **Obter URL da função** (Get function URL) no menu superior.
3. Copie essa URL e insira no arquivo index.html do Frontend para estabelecer a conexão entre os serviços.



APÊNDICE C

Módulo GCP: Projeção de Poder de Compra (IPCA)

Este guia detalha a criação da função *Serverless* no Google Cloud Platform utilizando o **Cloud Run functions**. Esta API é responsável por descontar uma taxa fictícia de inflação do montante final para projetar o poder de compra real.

Primeiro, crie um projeto:

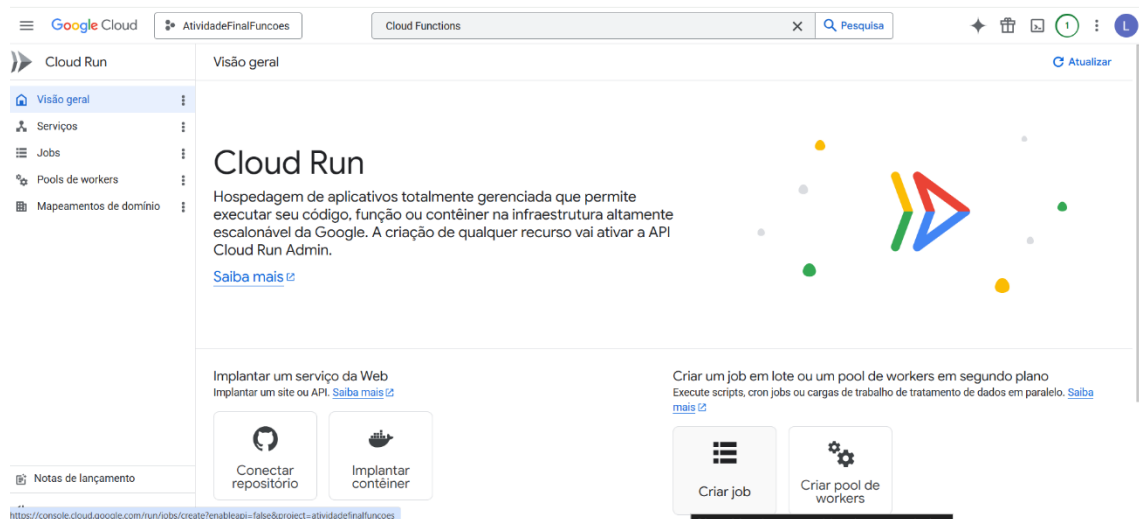
The screenshot shows the 'Novo projeto' (New Project) form in the Google Cloud console. The form includes the following fields and options:

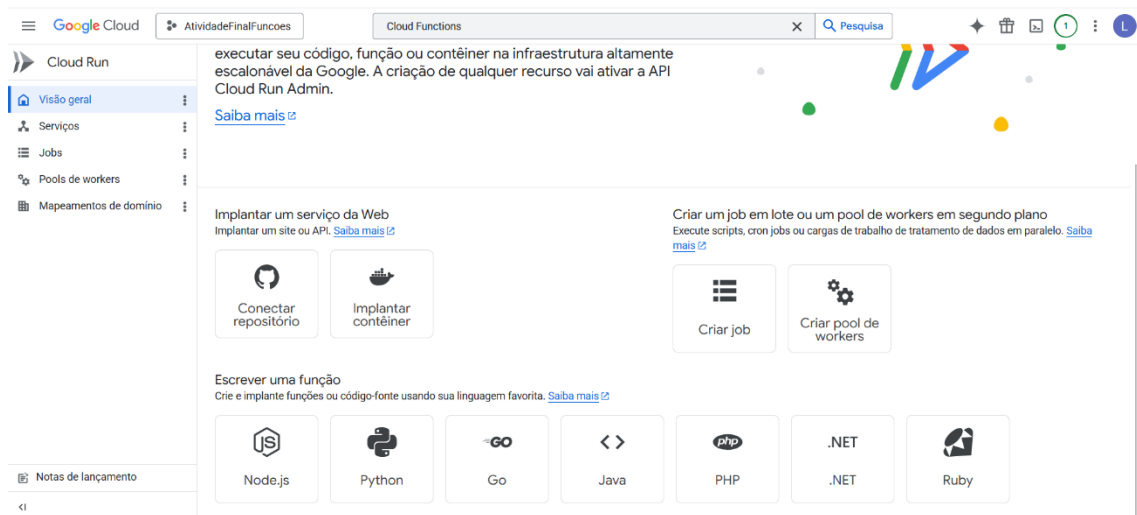
- Nome do projeto ***: AtividadeFinalFuncoes
- ID do projeto**: atividadefinalfuncoes. Não é possível mudar depois. [Editar](#)
- Conta de faturamento ***: Minha conta de faturamento
- Organização ***: leti-favoretto-org
- Recurso pai ***: leti-favoretto-org

Buttons at the bottom: **Criar** and **Cancelar**.

Passo 1: Acesso e Configuração Básica

1. Acesse o console do Google Cloud, pesquise por **Cloud Run functions** (ou Cloud Functions) na barra de pesquisa superior e clique no serviço.





2. Clique no botão **Criar função** (Create function).
3. Na seção Ambiente, escolha a **2ª geração** (2nd gen).
4. Na seção Gatilho (Trigger):
 - Escolha **HTTPS**.
 - Marque a opção **Permitir invocações não autenticadas** (Allow unauthenticated invocations).
 - **⚠️ Atenção: Isso é obrigatório para que o frontend público consiga acessar a API sem erros de permissão (401/403).**
5. Clique em **Avançar** (Next).

Escalonamento de serviços ?

☒ Escalonamento automático

Número mínimo de instâncias

0

Número máximo de instâncias

Defina como 1 para reduzir inicializações a frio. [Saiba mais](#)

☐ Escalonamento manual

Entrada ?

☐ Interno

Permitir o tráfego do projeto, da VPC compartilhada e do perímetro do VPC Service Controls. O tráfego de outro serviço do Cloud Run precisa ser roteado por uma VPC. Sujeito a limitações. [Saiba mais](#)

☐ Permitir tráfego de balanceadores de carga de aplicativo globais externos

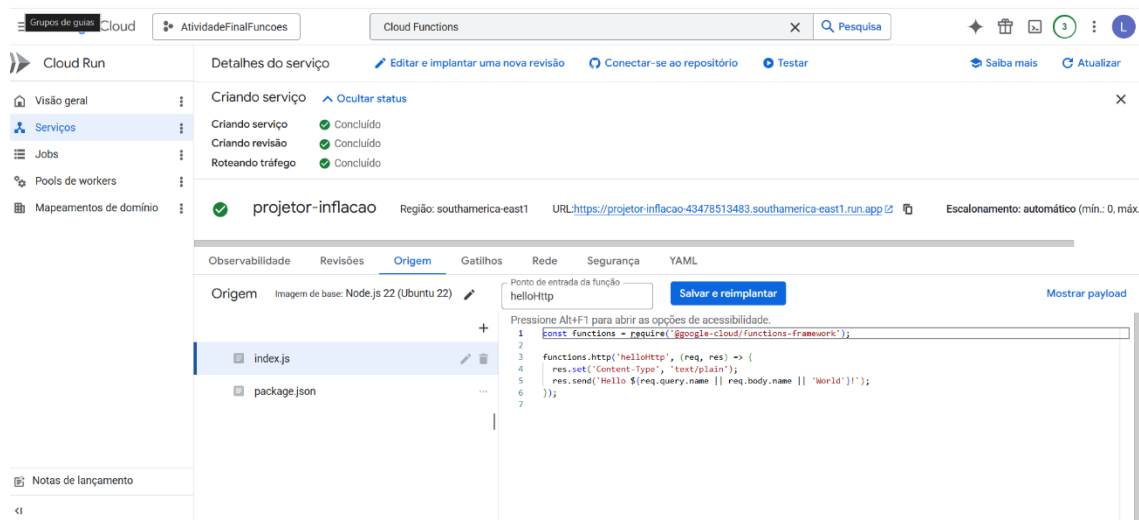
☒ Todos

Permita o acesso direto ao seu serviço pela Internet

Passo 2: O Código e o CORS (index.js)

Assim como na AWS, o Google Cloud exige que as regras de CORS (Cross-Origin Resource Sharing) sejam configuradas diretamente dentro do código Node.js.

1. Na etapa de código, mude o **Ambiente de execução** (Runtime) para **Node.js** (versão 20 ou superior).
2. No campo **Ponto de entrada** (Entry point), digite o nome exato da função exportada: `projetoInflacao`.
3. No editor de código, selecione o arquivo `index.js`, apague tudo e cole o código `projetoInflacao`



Passo 3: Dependências (package.json)

Para que a função entenda a biblioteca do Google Cloud que estamos utilizando no arquivo principal, precisamos declará-la nas dependências.

1. Ainda no editor de código, clique na aba do arquivo package.json.
2. Certifique-se de que o conteúdo esteja semelhante a este (especialmente a seção dependencies):

```
{  
  
  "name": "projektor-inflacao",  
  
  "version": "1.0.0",  
  
  "main": "index.js",  
  
  "dependencies": {  
  
    "@google-cloud/functions-framework": "^3.0.0"  
  
  }  
  
}
```

Passo 4: Implantação e Obtenção da URL

1. Clique no botão azul **Implantar** (Deploy) na parte inferior da tela.
2. Aguarde alguns minutos enquanto o Google Cloud provisiona o servidor e sobe o seu código. Um ícone de confirmação verde aparecerá quando terminar.
3. Clique no nome da sua função recém-implantada e vá até a aba **Gatilhos** (Triggers).
4. Copie a **URL do Gatilho** exibida na tela.
5. Cole essa URL na variável correspondente do painel web (index.html) para finalizar a integração do ecossistema!

APÊNDICE D

Módulo Frontend: Hospedagem e Deploy (Firebase Hosting)

Este guia documenta o processo de preparação do ambiente local e a implantação da interface web utilizando o Firebase Hosting para garantir um ambiente seguro e contornar restrições locais de CORS.

Passo 1: Preparação do Ambiente Local

- Instalação do **Node.js** para habilitação do gerenciador de pacotes (npm).
- Configuração de permissões de execução de scripts no Windows PowerShell (caso necessário) utilizando o comando abaixo:

Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

Passo 2: Instalação do CLI do Firebase

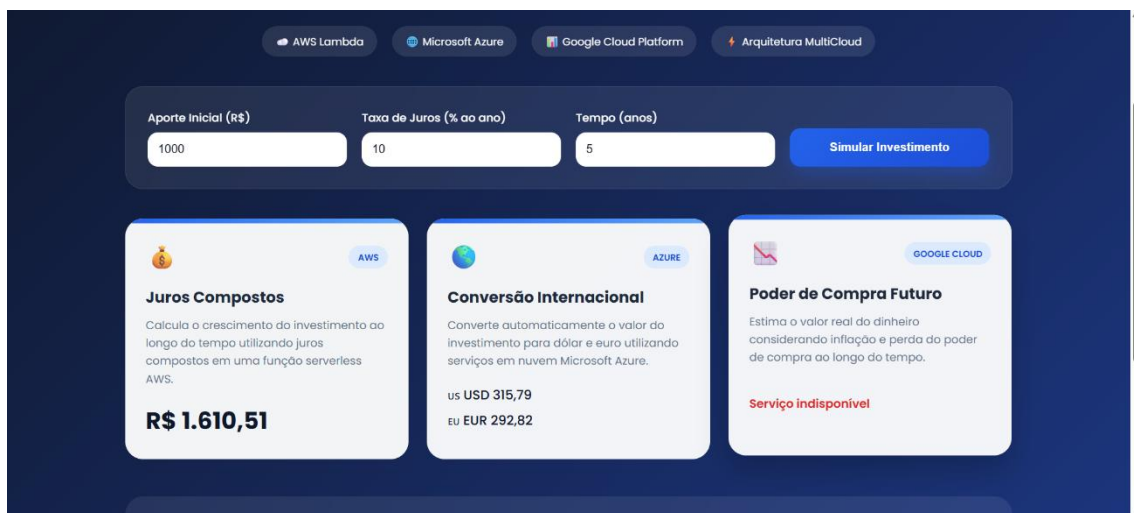
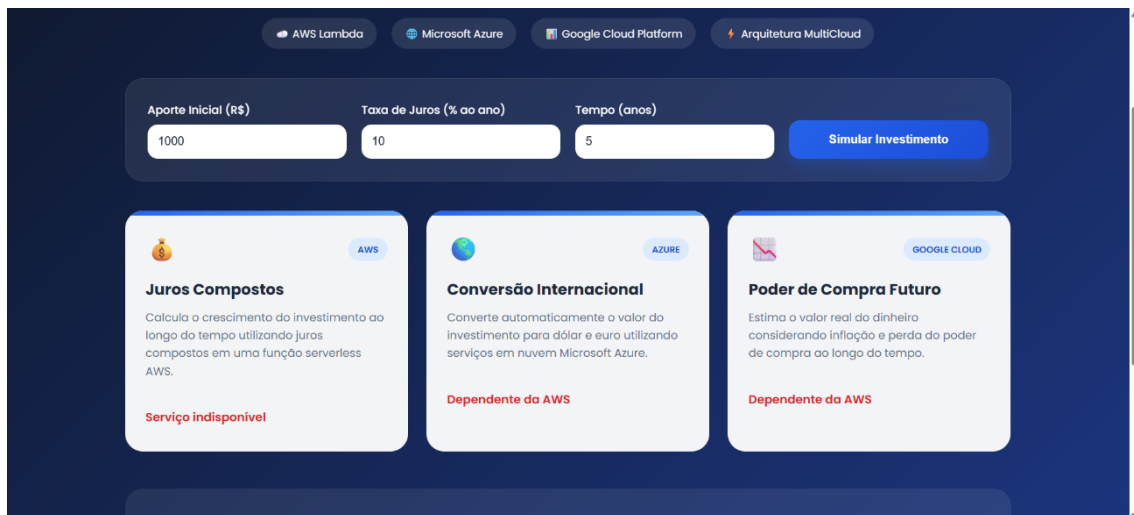
- Execução do comando de instalação global da ferramenta do Firebase via terminal do VSCode: **npm install -g firebase-tools**

Passo 3: Autenticação e Inicialização

- Vínculo da conta Google responsável pelo projeto através do comando: **firebase login**
- Criação do ecossistema de hospedagem na pasta raiz do projeto rodando: **firebase init hosting**
- Durante a inicialização, seguimos as seguintes definições interativas:
 - Vinculamos ao projeto já existente no Google Cloud.
 - Definimos a raiz pública como . (ponto).
 - Respondemos Não (N) para evitar sobrescrever o arquivo index.html original.

Passo 4: Deploy Final

- Envio dos arquivos de produção (HTML/CSS/JS) para os servidores globais do Google via comando final: **firebase deploy**
- Validação da funcionalidade através da URL segura (<https://seu-projeto.web.app>) gerada no console, sanando em definitivo os bloqueios de origem nula (null origin) impostos pelos navegadores durante os testes locais.



AWS Lambda

Microsoft Azure

Google Cloud Platform

Arquitetura MultiCloud

Aporte Inicial (R\$)

Taxa de Juros (% ao ano)

Tempo (anos)

Simular Investimento

Juros Compostos

Calcula o crescimento do investimento ao longo do tempo utilizando juros compostos em uma função serverless AWS.

R\$ 1.610,51

Conversão Internacional

Converte automaticamente o valor do investimento para dólar e euro utilizando serviços em nuvem Microsoft Azure.

Serviço indisponível

Poder de Compra Futuro

Estima o valor real do dinheiro considerando inflação e perda do poder de compra ao longo do tempo.

R\$ 1.279,32